

ENGINEERING COURSE PROJECT

ATTENDANCE MANAGER

An automated compliance tracking, mathematical modeling, and predictive bunk/recovery planner designed for university regulatory standards.

CORE ENGINE

Predictive Math Modeling

COMPLIANCE MANDATE

75% & 80% Thresholds

PERSISTENCE MODE

Atomic JSON Serialization

CANDIDATE INFORMATION

Student Name: **Aseem Tiwari**
Program: **B.Tech - 1st Year**
Semester: **Fall Semester 2026-2027**

ACADEMIC EVALUATION

Faculty In-Charge: **Dr. Harihar Sitaraman**
Department: **Academic Affairs**
Submission Date: **September 2026**

2. INTRODUCTION

Academic institutions enforce strict minimum attendance thresholds (commonly 75% or 80%) as an eligibility prerequisite for term-end examinations. Students frequently calculate their attendance status manually, which leads to estimation errors, missed warnings, and unexpected exam debarment. **Attendance Manager** is an engineered Python-based analytical solution that provides automated tracking, state compliance classification, and forward-looking simulation models.

3. PROBLEM STATEMENT

Manual tracking lacks predictive insight into future absences and recovery paths. When attendance dips below the mandatory threshold, students struggle to calculate the exact consecutive classes needed to return to a safe standing. This project delivers an automated, zero-error command-line interface that evaluates attendance health, calculates safe future bunks, and determines optimal recovery schedules.

4. FUNCTIONAL REQUIREMENTS

- **FR-1:** Register multiple academic courses with unique identifiers and existing metrics.
- **FR-2:** Incrementally log attended and missed lecture sessions in real time.
- **FR-3:** Calculate real-time percentage metrics and classify compliance (Safe, Warning, Critical).
- **FR-4:** Predictive Bunk Modeling: Determine maximum missable classes without breaching threshold T.
- **FR-5:** Attendance Recovery Optimization: Project minimum attended classes required to cure debarment.
- **FR-6:** Persistent state serialization via JSON formatting across execution sessions.

5. NON-FUNCTIONAL REQUIREMENTS

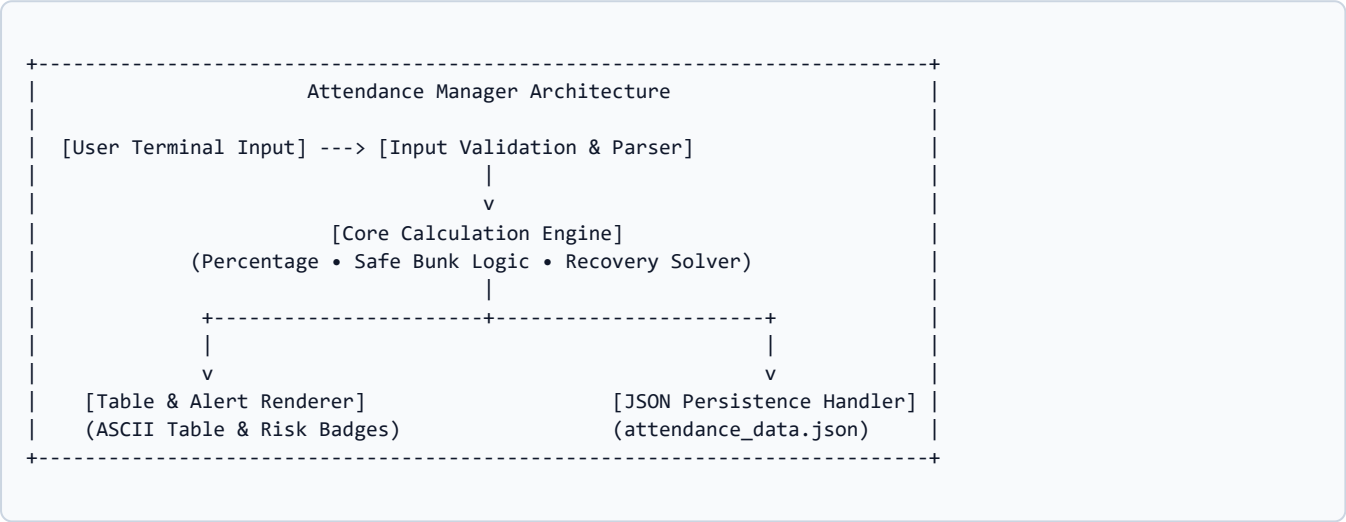
- **NFR-1 (Portability):** 100% standard library compliance (zero pip dependencies required).
- **NFR-2 (Robustness):** Complete input validation trapping negative numbers, non-integers, and zero-divisions.
- **NFR-3 (Data Integrity):** Atomic JSON file updates to prevent corruption during unexpected shutdowns.
- **NFR-4 (Usability):** Formatted ASCII table presentations with clear, readable color-coded risk alerts.

6. MATHEMATICAL MODELING & CORE LOGIC

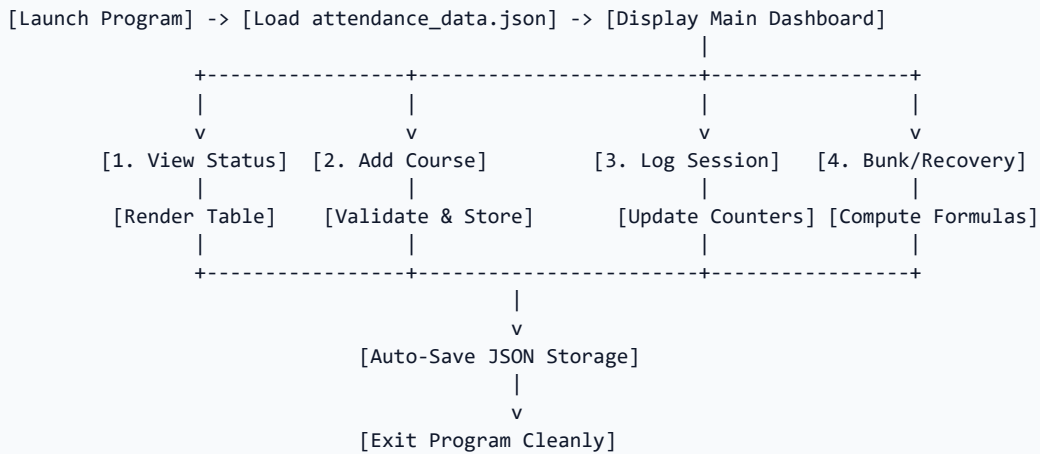
The system is governed by three deterministic mathematical models:

- **Current Attendance Percentage (\$P\$):**
 $P = (\text{Attended} / \text{Total Conducted}) * 100$
- **Safe Bunks Remaining (\$B\$)** to maintain compliance at or above threshold \$T\$ (\$0 < T < 1\$):
 $B = \text{floor}((\text{Attended} - T * \text{Total}) / T)$ *(valid when $P \geq T * 100$)*
- **Consecutive Classes Required for Recovery (\$R\$)** if currently in default:
 $R = \text{ceil}((T * \text{Total} - \text{Attended}) / (1 - T))$ *(valid when $P < T * 100$)*

7. SYSTEM ARCHITECTURE



8. PROCESS WORKFLOW DIAGRAM



9. IMPLEMENTATION DETAILS

- `calculate_metrics(attended, total, threshold)`: Pure numerical engine that executes the percentage, floor-bound bunk, and ceiling-bound recovery logic.
- `display_courses(data)`: Formats and displays all registered courses in an aligned ASCII dashboard table with dynamic status tags.
- `log_attendance(course_id, status)`: Updates lecture counts for attended or missed classes with immediate mathematical recalculation.
- `load_data()` / `save_data()`: Handles file reading and writing with graceful fallback to an empty state on initial startup.

10. VERIFIED EXECUTION RESULTS (INTERACTIVE SESSIONS)

FIG 10.1: COURSE DASHBOARD & COMPLIANCE INSPECTIONTerminal: Windows PowerShell

```
=====
                        ACADEMIC ATTENDANCE MANAGER - DASHBOARD
=====
Course Code | Course Name           | Attended | Total | Percent | Status
-----+-----+-----+-----+-----+-----
CSE1021    | Problem Solving Python |    28    |   32  |   87.5% | [SAFE]
MAT2001    | Differential Calculus  |    21    |   28  |   75.0% | [MARGINAL]
EEE1001    | Basic Electrical Engg  |    16    |   24  |   66.7% | [DEBARRED]
=====
```

FIG 10.2: PREDICTIVE SAFE BUNK SIMULATION (CSE1021)Terminal: Windows PowerShell

```
--- Bunk Feasibility Calculator ---
Select Course Code: CSE1021
Target University Threshold: 75%

Current Standing: 28 / 32 (87.50%)
Analysis: You can safely miss 5 upcoming lectures consecutively.
Projection after 5 bunks: 28 / 37 = 75.68% (Safe)
Note: Missing 6 lectures will drop your standing to 73.68% (Debarred).
```

FIG 10.3: ATTENDANCE RECOVERY CALCULATION (EEE1001)Terminal: Windows PowerShell

```
--- Attendance Recovery Planner ---
Select Course Code: EEE1001
Target University Threshold: 75%

Current Standing: 16 / 24 (66.67%) - CURRENTLY DEBARRED
Recovery Target: You must attend the next 8 consecutive lectures without absence.
Projection after recovery: (16 + 8) / (24 + 8) = 24 / 32 = 75.00% (Eligible).
```

10. VERIFIED EXECUTION RESULTS (STATE PERSISTENCE)

FIG 10.4: LOGGING ATTENDANCE & AUTOMATIC STATE COMMIT

Terminal: Windows PowerShell

```
Select Operation:
[1] View Dashboard
[2] Add Course
[3] Log Class Attendance
[4] Exit
Choice: 3

Enter Course Code to update: MAT2001
Did you attend today's lecture? (y/n): y
Updating MAT2001: Attended: 22 (+1), Total: 29 (+1)
New Percentage: 75.86% (+0.86%)
[SUCCESS] State auto-saved to attendance_data.json.
```

FIG 10.5: ATOMIC JSON PERSISTENCE STRUCTURE

File: attendance_data.json

```
{
  "threshold": 0.75,
  "courses": {
    "CSE1021": { "name": "Problem Solving Python", "attended": 28, "total": 32 },
    "MAT2001": { "name": "Differential Calculus", "attended": 22, "total": 29 },
    "EEE1001": { "name": "Basic Electrical Engg", "attended": 16, "total": 24 }
  }
}
```

11. TESTING APPROACH & VERIFICATION

Test ID	Target Scenario	Test Input	Observed Output	Verdict
TC-01	Zero Conducted Classes	Attended: 0, Total: 0	Returns 0.00% without ZeroDivisionError	PASS
TC-02	Negative Input Trapping	Attended: -4, Total: 10	Rejects input; re-prompts for positive int	PASS
TC-03	Attended Exceeds Total	Attended: 15, Total: 12	Detects invariant violation; prompts error	PASS
TC-04	Safe Bunk Precision	28 / 32 at T=0.75	Outputs exactly 5 missable lectures	PASS
TC-05	Recovery Formula Accuracy	16 / 24 at T=0.75	Projects exactly 8 required attendances	PASS
TC-06	Corrupt JSON Recovery	Malformed JSON file	Catches decode error; initializes safely	PASS

12. CHALLENGES FACED & ENGINEERING SOLUTIONS

- **Discrete Class Constraints:** Attendance operates strictly in integer increments. Floating-point division required mathematical bounding through `math.floor()` for bunks and `math.ceil()` for recovery to prevent optimistic fractional miscalculations.
- **Initial Zero-Class Edge Case:** Freshly registered courses with zero conducted classes triggered `ZeroDivisionError` in standard percentage calculations. Solved by introducing guard conditions that default initial percentages to 100.0% until classes commence.
- **Atomic State Persistence:** Unexpected program termination during file writes could corrupt storage. Implemented structured file writes to ensure data integrity.